

Organização de sistemas de computadores

Parte 2



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

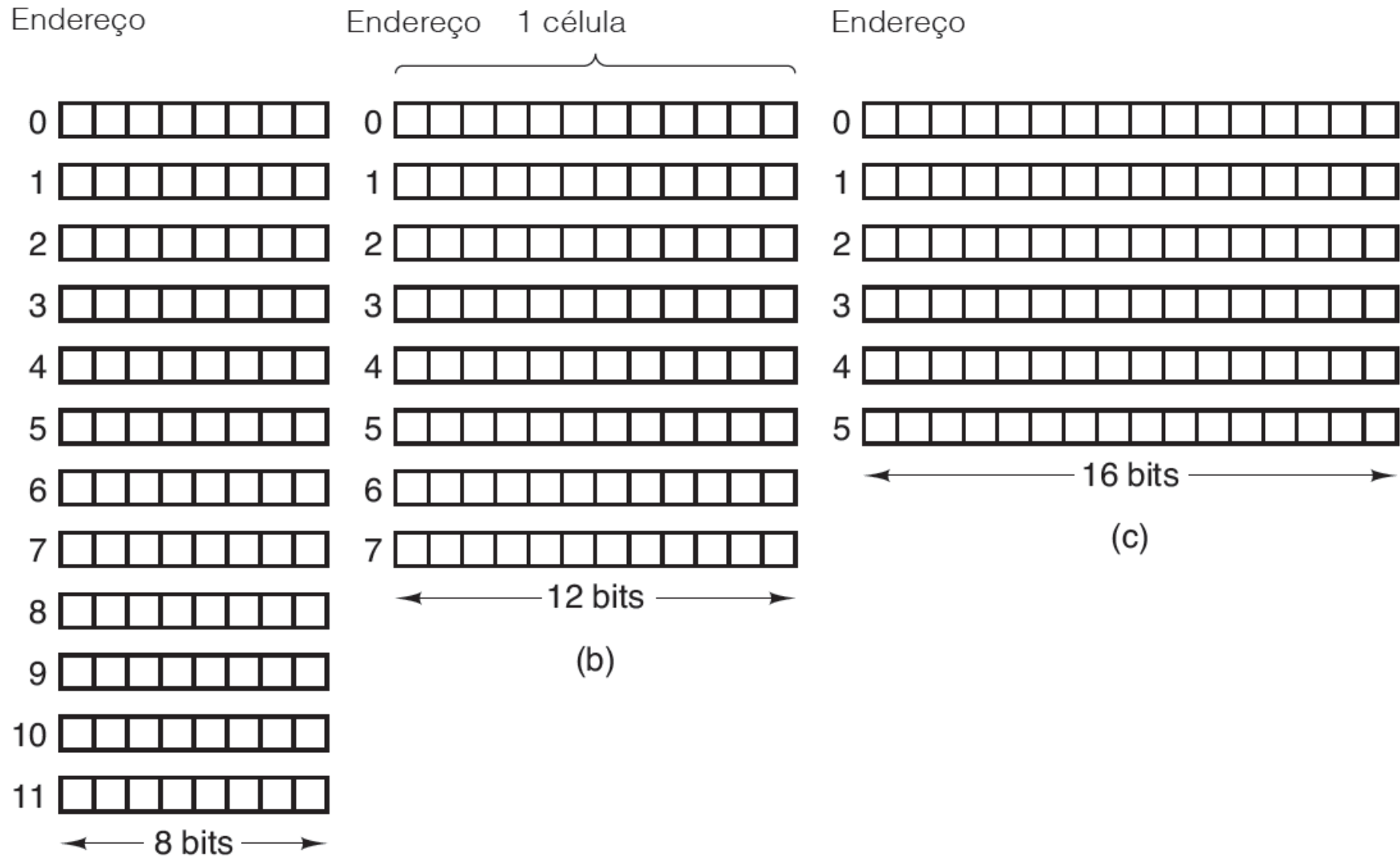
Prof. Carlos Minoru Tamaki

Memória primária

- A **memória** é a parte do computador onde são armazenados programas e dados.
- A unidade básica de memória é dígito binário, denominado **bit**. Um bit pode conter um 0 ou um 1.
- Memórias consistem em uma quantidade de **células** (ou **locais**), cada uma das quais podendo armazenar uma informação.
- Cada célula tem um número, denominado seu **endereço**, pelo qual os programas podem se referir a ela.

Memória primária

- Três maneiras de organizar uma memória de 96 bits.

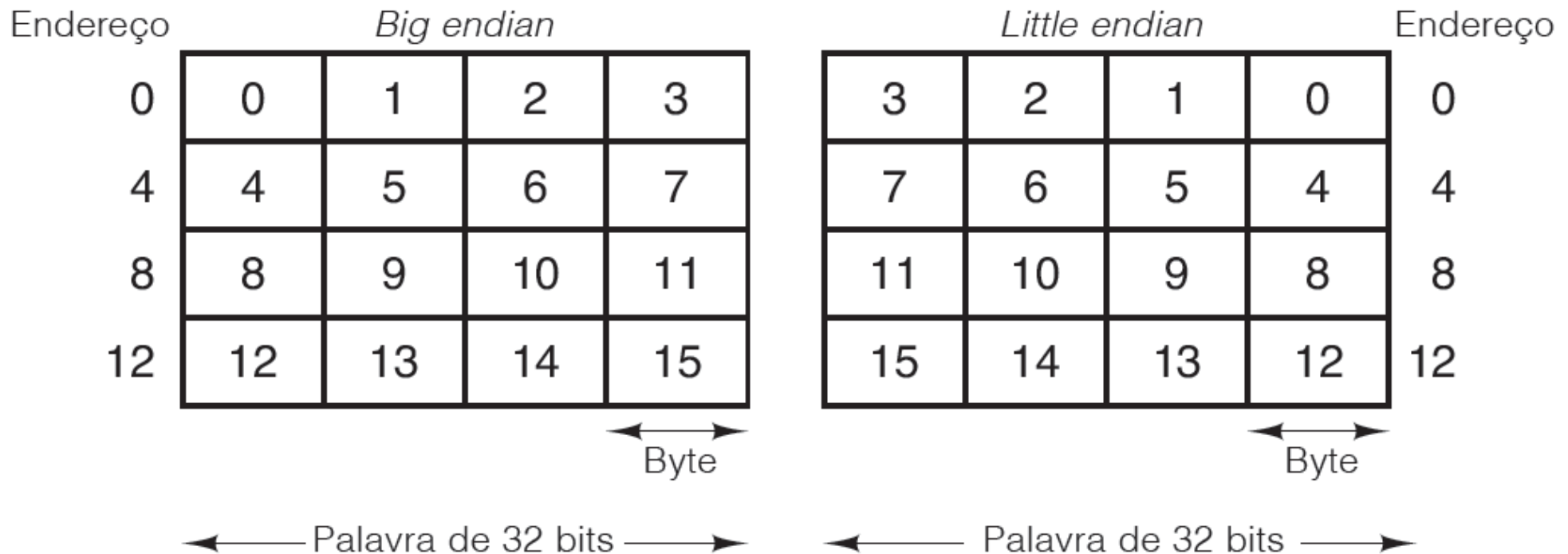


Computador	Bits/célula
Burroughs B1700	1
IBM PC	8
DEC PDP-8	12
IBM 1130	16
DEC PDP-15	18
XDS 940	24
Electrologica X8	27
XDS Sigma 9	32
Honeywell 6180	36
CDC 3600	48
CDC Cyber	60

Número de bits por célula para alguns computadores comerciais historicamente interessantes.

Memória primária

- Os **bytes** em uma palavra podem ser numerados da esquerda para a direita ou da direita para a esquerda.
- Memória *big endian* e memória *little endian*.



	Big endian			
0	J	I	M	
4	S	M	I	T
8	H	0	0	0
12	0	0	0	21
16	0	0	1	4

(a)

**Registro Pessoal
para uma máquina
*big endian***

	Little endian			
0		M	I	J
4	T	I	M	S
8	0	0	0	H
12	0	0	0	21
16	0	0	1	4

(b)

**O mesmo registro
para uma máquina
*little endian***

Trasnferência de
big endian para little
endian

0		M	I	J
4	T	I	M	S
8	0	0	0	H
12	21	0	0	0
16	4	1	0	0

(c)

**Resultado da
trânsferência de
big endian para
*little endian***

Transferência
e deslocamento

0	J	I	M	
4	S	M	I	T
8	H	0	0	0
12	0	0	0	21
16	0	0	1	4

(d)

**Resultado do
deslocamento de
bytes (c)**

Códigos de correção de erro

- Memórias de computador podem cometer erros de vez em quando devido a picos de tensão na linha elétrica, raios cósmicos ou outras causas.
- As propriedades de detecção de erro e correção de erro de um código dependem de sua distância de Hamming.
- Para detectar d erros de único bit, você precisa de um código de distância $d + 1$.
- Para corrigir erros de único bit, você precisa de um código de distância $2d + 1$.
- Bit de paridade devem ser acrescentados.

Códigos de correção de erro

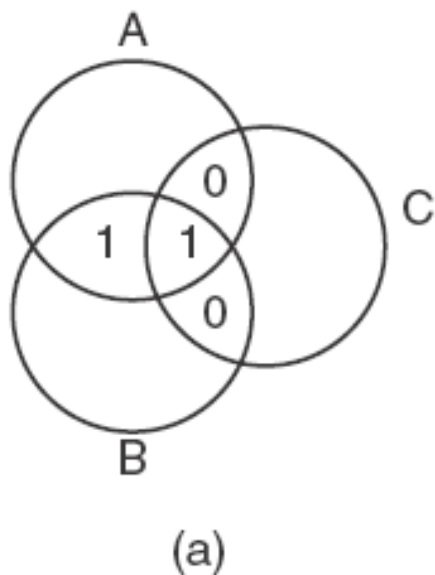
- Palavra de código $\Rightarrow n$ (número de bits total) = m (tamanho da palavra) + r (bits de redundância)
- Distância de Hamming é calculada executando-se um OU-EXCLUSIVO entre 2 palavras e contar os bits 1 gerados, por exemplo: 10001001 e 10110001, distam 3 unidades de Hamming.

$$\begin{array}{r} 10001001 \\ \oplus 10110001 \\ \hline 00111000 \end{array}$$

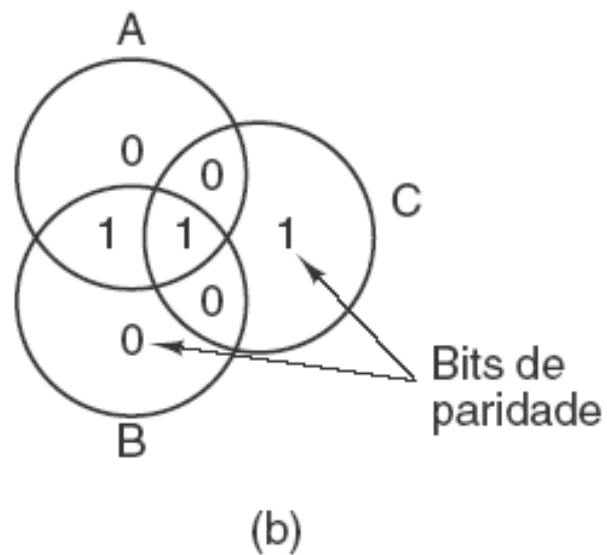
Códigos de correção de erro

- Número de bits de verificação para um código que pode corrigir um erro único.

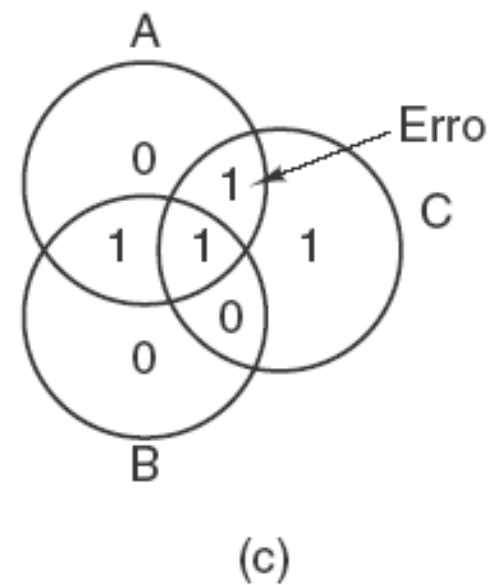
Tamanho da palavra	Bits de verificação	Tamanho total	Acréscimo percentual
8	4	12	50
16	5	21	31
32	6	38	19
64	7	71	11
128	8	136	6
256	9	265	4
512	10	522	2



Codificação de 1100



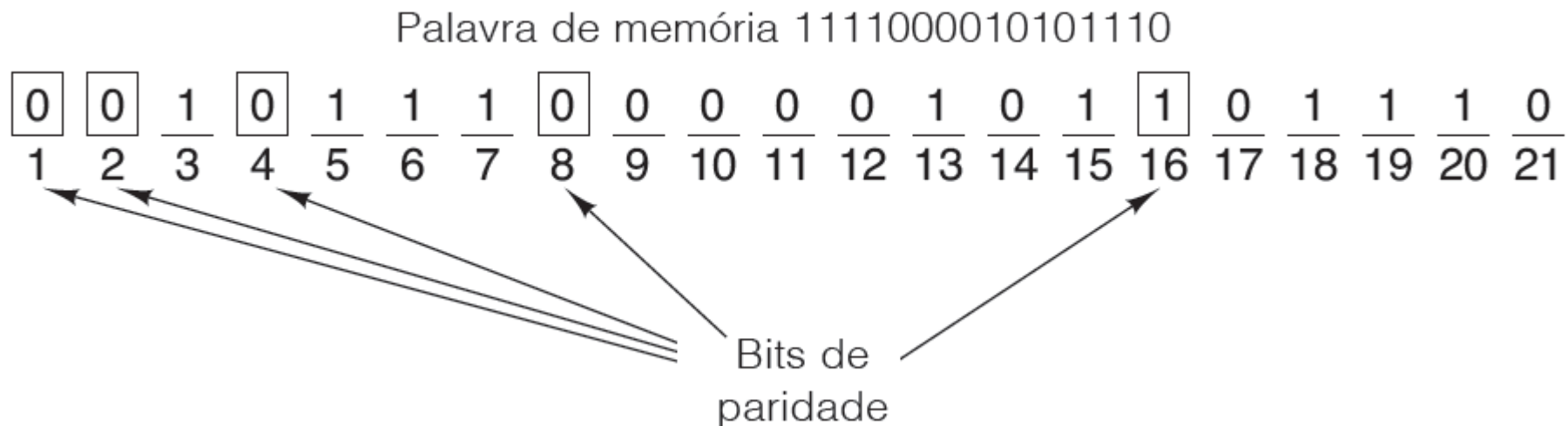
Paridade par adicionada



Erro em AC

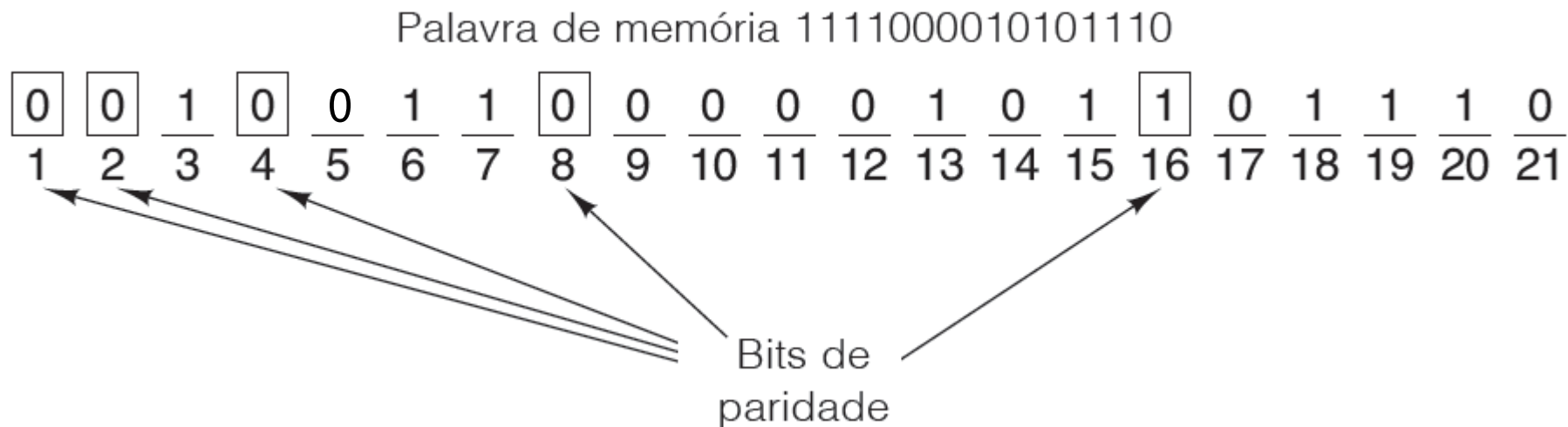
Códigos de correção de erro

- A figura a seguir mostra a construção de um código de Hamming para a palavra de memória de 16 bits 1111000010101110.



- A palavra de código de 21 bits é 001011100000101101110.
- Para ver como funciona a correção de erros, considere o que aconteceria se o bit 5 fosse invertido por uma sobrecarga elétrica na linha de força.
- A nova palavra de código seria 001001100000101101110 em vez de 001011100000101101110.

Códigos de correção de erro



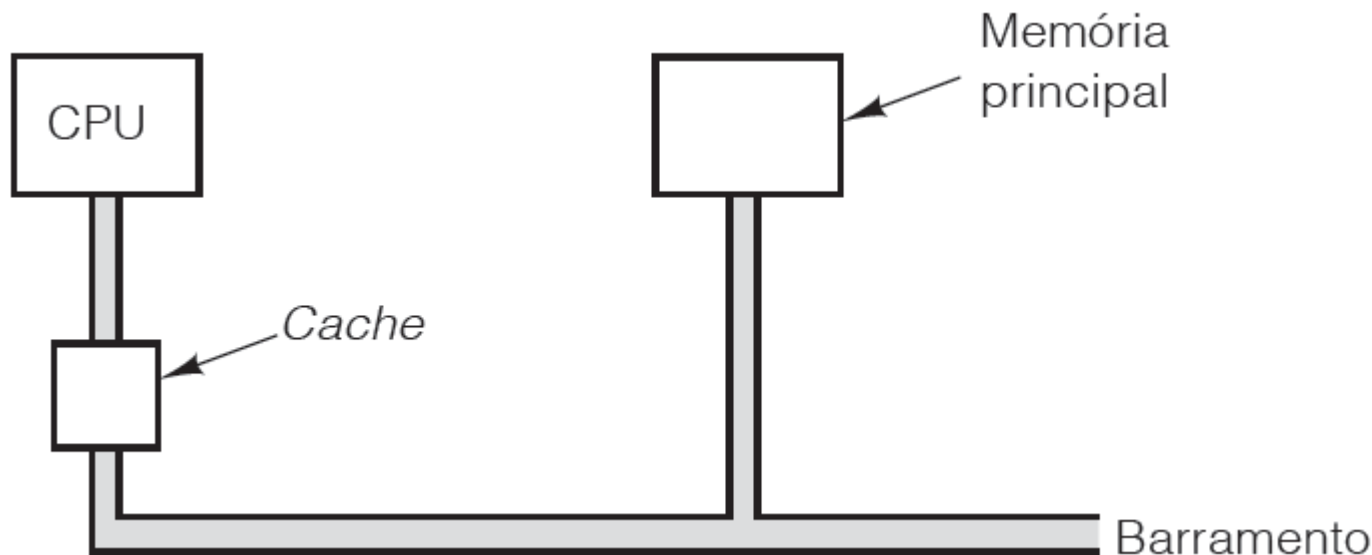
- **Os 5 bits de paridade serão verificados com os seguintes resultados:**
- Bit de paridade 1 incorreto (1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21 contêm cinco 1s).
- Bit de paridade 2 correto (2, 3, 6, 7, 10, 11, 14, 15, 18, 19 contêm seis 1s).
- Bit de paridade 4 incorreto (4, 5, 6, 7, 12, 13, 14, 15, 20, 21 contêm cinco 1s).
- Bit de paridade 8 correto (8, 9, 10, 11, 12, 13, 14, 15 contêm dois 1s).
- Bit de paridade 16 correto (16, 17, 18, 19, 20, 21 contêm quatro 1s).

Memória *cache*

- A ideia básica de uma *cache* é simples: as palavras de memória usadas com mais frequência são mantidas na *cache*.
- Quando a CPU precisa de uma palavra, ela examina em primeiro lugar a *cache*. Somente se a palavra não estiver ali é que ela recorre à memória principal.
- Se uma fração substancial das palavras estiver na *cache*, o tempo médio de acesso pode ser muito reduzido.

Memória *cache*

- A localização lógica da *cache* é entre a CPU e a memória principal.
- Em termos físicos, há diversos lugares em que ela poderia estar localizada.



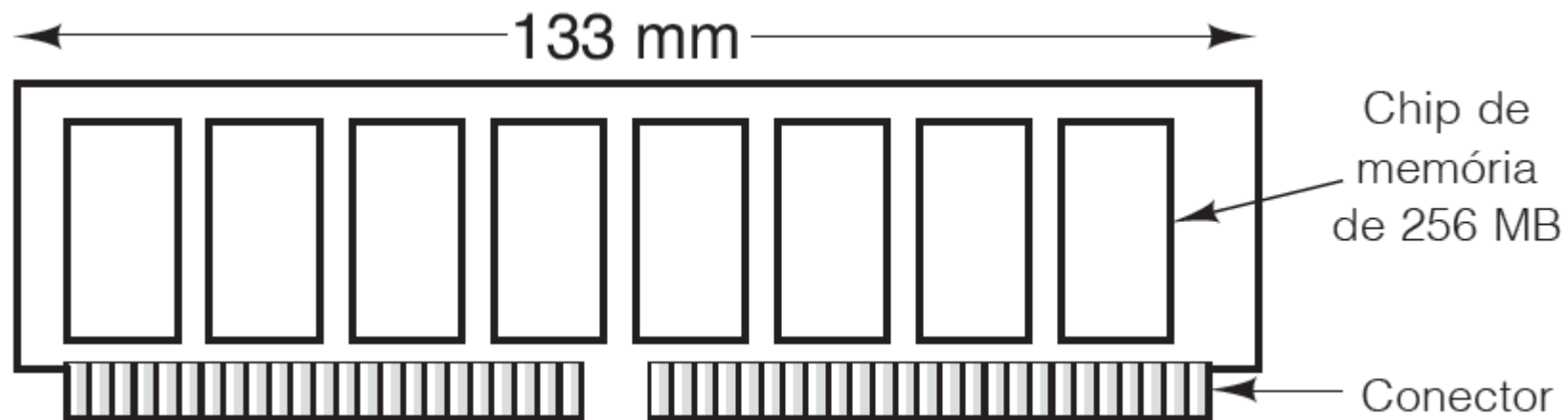
- Procura-se beneficiar do **PRINCÍPIO DA LOCALIDADE.**

Memória *cache*

- Quantidade de vezes que um determinado dado é acessado – k
- Taxa de acertos: representa a fração de todas as referências que puder ser satisfeitas pela cache – h
- Taxa de falhas: $1 - h$
- Tempo médio de acesso = $c + (1 - h).k$
- Onde c é o tempo de acesso da memória cache

Empacotamento e tipos de memória

- Uma configuração típica de DIMM poderia ter oito chips de dados com 256 MB cada. Então, o módulo inteiro conteria 2 GB.
- Muitos computadores têm espaço para quatro módulos, o que dá uma capacidade total de 8 GB se usarem módulos de 2 GB e mais, se usarem módulos maiores.





DDR4 Notebook



DDR2 Desktop



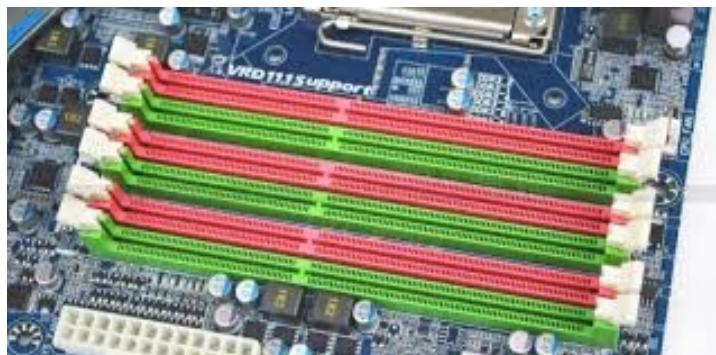
DDR3 Desktop



DDR4 com dissipador



DDR5 32GB/pente



Slots de memória
na placa mãe



Evolução das memórias

Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall
- Arquitetura e Organização de Computadores – William Stallings – 11ª Edição 2024 Prentice Hall
- Diversos sites da internet para imagens